SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**
Question Count: 5

Code of Conduct

I Athavan K (192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Athavan k

Assessment Tasks Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design	25%	Design is robust,	Reasonable	Basic design	Weak design

Constraint-Based Problem Solving Rubric

Quality		feasible, and aligned to constraints	design	with gaps	
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, wellstructured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Question 1 — Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

1. Constraints Identified and Prioritized

- **Availability > 99.9% (highest priority):** the platform cannot tolerate more than ~8.7 hours of downtime per year, so single points of failure (NameNode, rack, network switch) must be eliminated.
- **Replication factor ≤ 3:** durability and read throughput must be achieved without exceeding 3 copies per block, capping raw storage overhead at 3×.
- **Cost-efficient storage growth:** media files (video/images) are large and grow continuously, so the design must reduce long-term storage cost without breaching the first two constraints.

2. Proposed Architecture

The design uses HDFS High Availability (HA) with a hot/cold tiering strategy layered on top of rack-aware replication, so that the three constraints are satisfied simultaneously rather than traded off against each other.

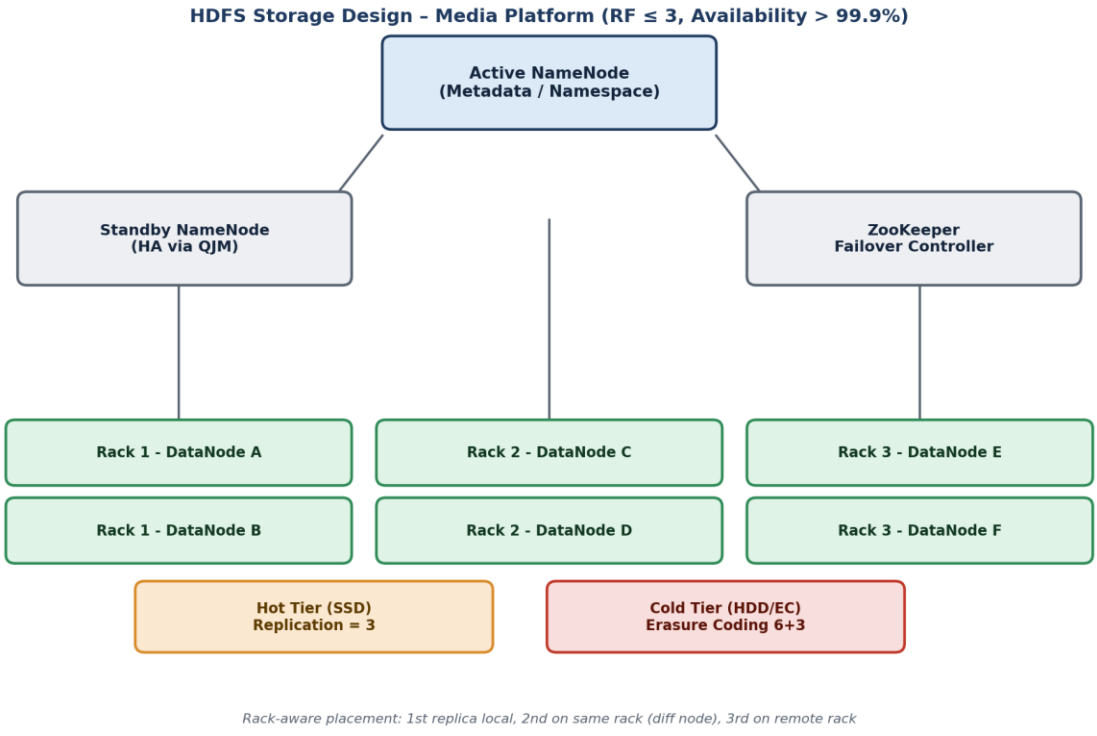

Figure 1: HDFS storage design for the media platform showing NameNode HA, rack-aware DataNode placement, and hot/cold tiering.

3. Design Components and Justification

- **Active/Standby NameNode with Quorum Journal Manager (QJM):** eliminates the single point of failure at the metadata layer. Automatic failover via ZooKeeper Failover Controller (ZKFC) keeps metadata service available within seconds of a crash, directly satisfying the 99.9% availability constraint.
- **Rack-aware replication (RF = 3):** the default HDFS placement policy stores the first replica on the writer's node, the second on a different node in the same rack, and the third on a node in a different rack. This tolerates a full rack failure (power/switch) while staying at the maximum allowed RF of 3, so no additional storage cost is incurred.
- **Hot/Cold tiering for cost efficiency:** recently uploaded or trending media (hot data) is kept on SSD-backed DataNodes with RF = 3 for fast delivery; older, rarely accessed media (cold archive) is converted to Erasure Coding (EC 6+3), which reduces storage overhead from 200% (3× replication) to about 50% while still tolerating up to 3 simultaneous block failures.
- **HDFS Federation (optional scale-out):** as the media catalogue grows, multiple NameNodes can each own a namespace (e.g., videos, thumbnails, metadata), avoiding a single NameNode becoming a metadata bottleneck without breaking HA.
- **Balancer and heterogeneous storage policies:** the HDFS Balancer redistributes blocks across DataNodes to prevent hotspotting, and Archival Storage policies automatically migrate cold blocks to cheaper disks, keeping growth cost-efficient over time.

4. How the Design Satisfies Every Constraint

- Availability > 99.9%: achieved through NameNode HA + ZKFC automatic failover + rack-aware replica placement tolerating node/rack failure.
- Replication factor ≤ 3: hot tier uses exactly RF = 3 (the maximum allowed); cold tier switches to Erasure Coding, which is not bound by the replication-factor constraint since it uses parity blocks, not replica copies.
- Cost-efficient growth: EC on cold data plus automated tiering keeps the storage-cost curve sub-linear as the media library expands, instead of growing at a flat 3× multiplier.

Question 2 — A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

1. Constraints Identified and Prioritized

- **Hard time SLA (20 minutes):** the entire job — map, shuffle, reduce, and write-back — must complete within the batch window, every run, even under data-volume spikes.
- **Limited cluster nodes:** the number of parallel map/reduce slots is constrained, so task granularity and scheduling must be tuned instead of simply adding more machines.
- **Correctness and fault tolerance:** a node failure during the batch must not push the job past the SLA, since log data still needs to be fully processed.

2. Job Design

The job is designed to maximize node utilization inside a fixed time budget by controlling input-split size, using combiners to shrink shuffle traffic, tuning the map:reduce task ratio to the available slots, and enabling speculative execution to absorb straggler tasks.

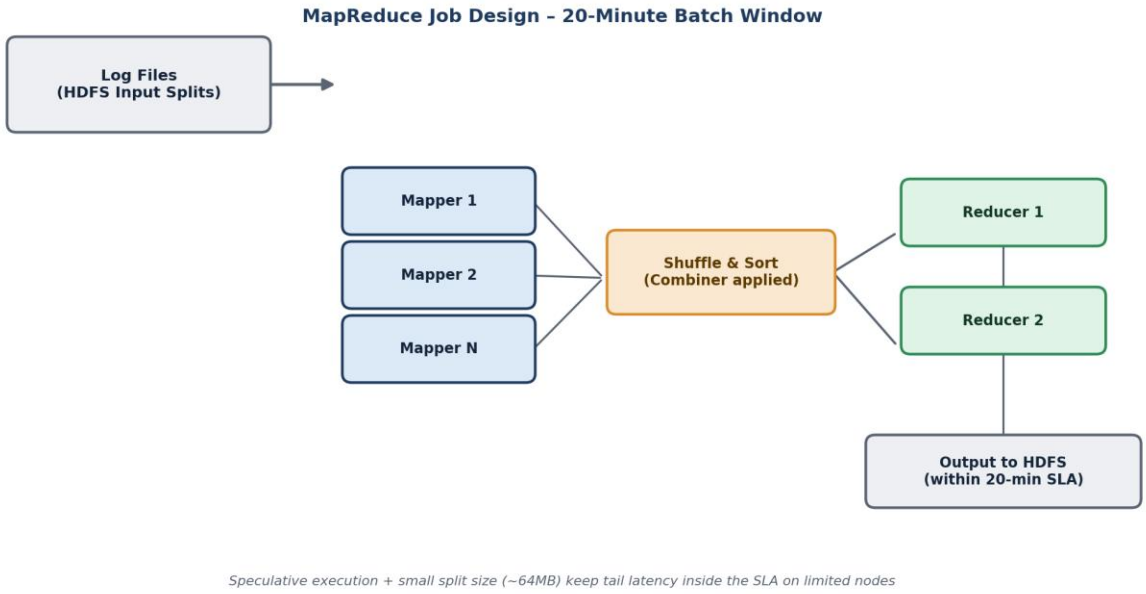

Figure 2: Task distribution for the log-processing MapReduce job within the 20-minute batch window.

3. Task Distribution and Justification

- **Smaller input splits (≈64 MB instead of 128 MB):** with limited nodes, more numerous, shorter-running map tasks give the scheduler finer-grained control, so if one task straggles, its impact on the total runtime is small and it can be re-launched quickly on another node.
- **Combiner before shuffle:** a local combiner performs partial aggregation (e.g., per-mapper log-count summation) on each node before data is sent over the network, cutting shuffle volume significantly and directly reducing the shuffle phase, which is usually the biggest risk to the SLA.
- **Reducer count tuned to cluster capacity:** the number of reducers is set close to the number of available reduce slots (rather than a large default), so all reducers start immediately instead of running in waves, keeping the reduce phase inside the remaining time budget.
- **Speculative execution enabled:** if a task runs noticeably slower than its peers (e.g., due to a degraded node), the framework proactively launches a duplicate on another node and keeps whichever finishes first — this protects the fixed 20-minute deadline from a single slow node.
- **Custom partitioner for even key distribution:** log keys (e.g., by server ID or error type) are partitioned to avoid skew, so no single reducer receives a disproportionate share of records that could blow the time budget.
- **Compression on intermediate data:** map output is compressed (e.g., Snappy) before shuffle to reduce network and disk I/O, which is often the dominant cost when the cluster is small.

4. How the Design Meets the Constraints

- 20-minute SLA: met by shrinking the shuffle payload (combiner + compression), avoiding reducer starvation (right-sized reducer count), and neutralizing stragglers (speculative execution).
- Limited nodes: met by increasing task parallelism through smaller splits rather than depending on additional hardware, so the existing slots are used with minimal idle time.

Question 3 — A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

1. Constraints Identified and Prioritized

- **Priority for ETL:** ETL jobs feed all downstream reporting and ML work, so they must get resources first and should be able to preempt lower-priority jobs when the cluster is saturated.
- **Fairness across workloads:** reporting and ML jobs must still receive a guaranteed minimum share of the cluster so they are never fully starved by ETL bursts.
- **Recovery from node failure:** a NodeManager crash must not fail running applications outright; the ResourceManager must reschedule affected containers with minimal disruption.

2. Scheduler Policy Design

The design uses YARN's Capacity Scheduler configured with three dedicated queues — ETL, Reporting, and ML — each with a guaranteed minimum capacity, elastic maximum capacity, and queue-level priority, combined with ResourceManager High Availability and NodeManager health monitoring for failure recovery.

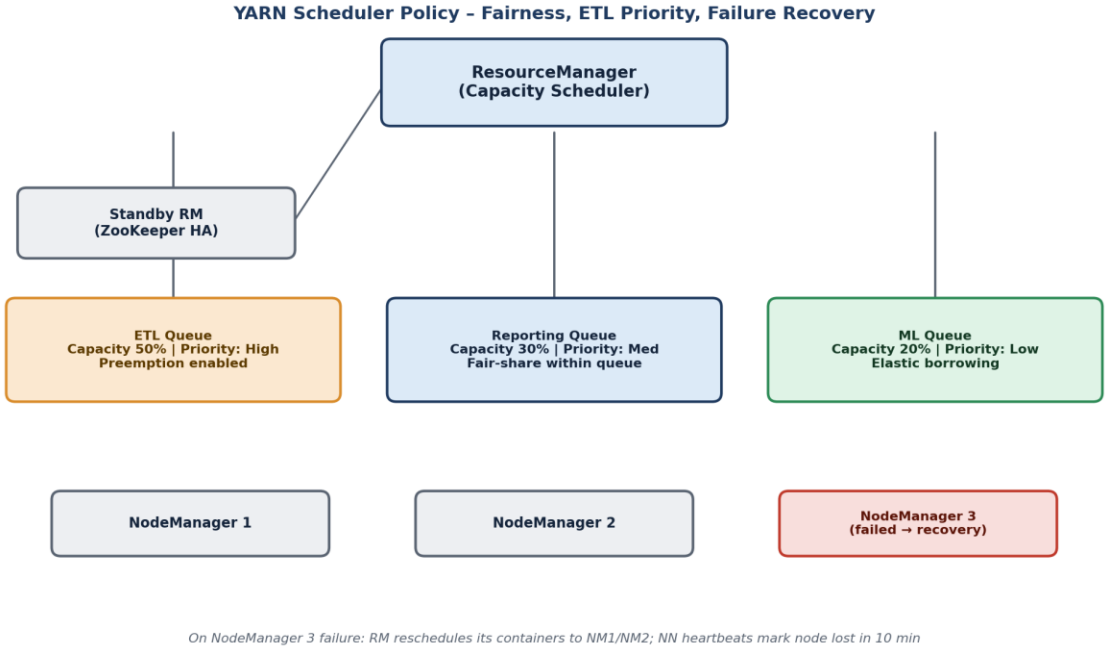

Figure 3: YARN Capacity Scheduler queue design balancing ETL priority, fairness, and failure recovery.

3. Queue Configuration and Justification

- **ETL Queue — 50% guaranteed capacity, High priority, preemption enabled:** guarantees ETL jobs a large baseline share and allows YARN to preempt containers from lower-priority queues when ETL demand spikes, satisfying the 'priority for ETL' constraint directly.
- **Reporting Queue — 30% guaranteed capacity, Medium priority, intra-queue fair scheduling:** protects interactive/scheduled reporting jobs from starvation; within the queue, a Fair Scheduler-style policy shares resources evenly among concurrent reporting jobs.
- **ML Queue — 20% guaranteed capacity, Low priority, elastic borrowing:** ML training jobs are typically longer and more tolerant of delay, so they get the smallest guaranteed share but can elastically borrow idle capacity from ETL/Reporting when those queues are underused, improving overall cluster utilization without breaking fairness.
- **Preemption policy with grace period:** when ETL needs more resources, the scheduler preempts the youngest containers in lower-priority queues first (not the oldest, to avoid wasting completed work) and gives a short grace period for checkpointing, balancing priority with fairness.

4. Node-Failure Recovery Design

- **ResourceManager High Availability (Active/Standby via ZooKeeper):** protects the scheduler itself from being a single point of failure.
- **NodeManager heartbeat and health checks:** the RM marks a node as lost if it misses heartbeats beyond a configured timeout (commonly around 10 minutes, tunable lower for this cluster), and immediately reschedules that node's containers onto healthy NodeManagers.
- **Application Master (AM) restart:** if the failed node was hosting an AM, YARN restarts the AM (up to a configured retry count) on another node; MapReduce/Spark jobs resume from the last completed task rather than restarting entirely, minimizing SLA impact.

- **Work-preserving recovery:** containers on surviving nodes are not killed when only the RM or an AM restarts, so healthy work in progress is preserved during recovery.

5. How the Design Meets the Constraints

- Fairness: each workload has a guaranteed minimum queue capacity, so no queue is ever starved even under contention.
- ETL priority: the largest guaranteed capacity plus preemption rights ensure ETL is served first without permanently blocking the other queues.
- Node-failure recovery: RM HA, heartbeat-based node-loss detection, and AM/container rescheduling keep jobs running with minimal manual intervention.

Question 4 — An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

1. Constraints Identified and Prioritized

- **Heterogeneous sources:** SAN (block storage), NAS (file shares), and operational databases (RDBMS/OLTP) each need a different ingestion mechanism.
- **Backup reliability:** ingested data must be recoverable even if the ingestion pipeline or primary cluster fails.
- **Minimal duplication:** the same record or file must not be re-ingested and stored multiple times, which would waste storage and corrupt downstream analytics.

2. Proposed Architecture

The architecture routes each source type through the connector best suited to it, funnels all of them through a single Apache NiFi ingestion layer for governance and deduplication, and lands data in a zoned HDFS structure that is protected by scheduled distributed-copy backups.

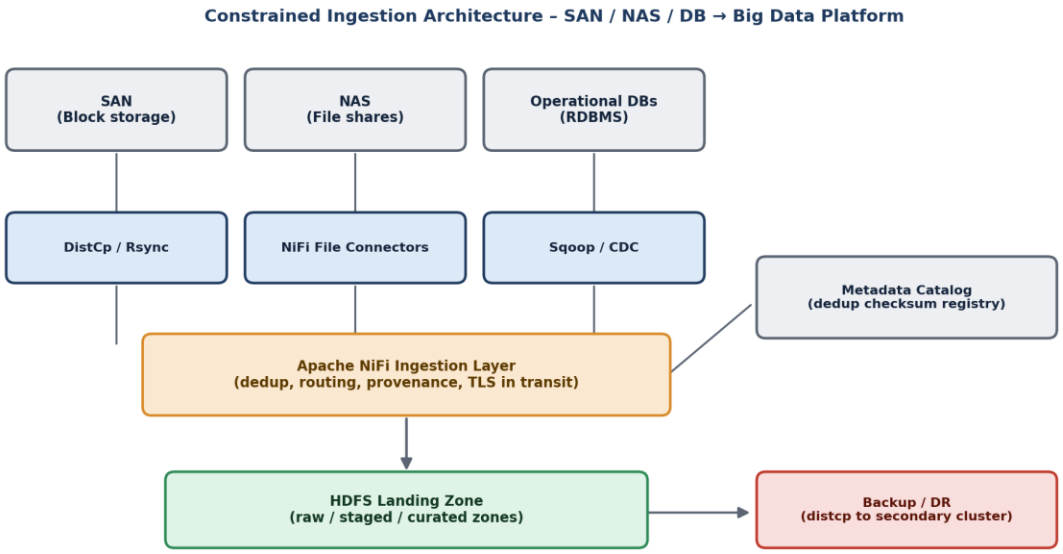

Figure 4: Constrained ingestion architecture unifying SAN, NAS, and database sources with backup and deduplication controls.

3. Component Justification

- **DistCp / Rsync for SAN:** SAN exposes block-level storage typically mounted as a filesystem to a host; DistCp (for HDFS-to-HDFS style transfer after mounting) or Rsync efficiently copies large block-derived files while preserving checksums, which supports minimal duplication through checksum-based skip-if-unchanged transfers.
- **NiFi File/SFTP Connectors for NAS:** NAS is file-share-based, so NiFi's GetFile/ListFile and SFTP processors natively poll shares, track already-seen files (via NiFi's state management), and avoid re-pulling files that have already been ingested.
- **Sqoop / Change-Data-Capture (CDC) for operational databases:** Sqoop performs efficient bulk/incremental imports using primary-key or timestamp watermarks, while CDC (e.g., Debezium-style log-based capture) streams only row-level changes; both avoid re-importing unchanged rows, which is the main duplication risk for OLTP sources.
- **Central Apache NiFi ingestion layer:** acts as the single governance point for all three feeds — deduplicating by content checksum/record key, enforcing TLS-encrypted transit for security, and recording full data provenance (lineage of every ingested record), which is essential for audit and troubleshooting.
- **Metadata catalog with checksum registry:** before NiFi commits a file or record batch to HDFS, it checks the catalog for a matching checksum/key; a match is skipped, directly enforcing the 'minimal duplication' constraint across all three source types uniformly.
- **Zoned HDFS landing area (raw / staged / curated):** raw preserves an untouched copy of ingested data for lineage and re-processing; staged applies cleaning/validation; curated holds analytics-ready data — this separation prevents accidental overwrites and duplicate curation runs.
- **DistCp-based backup to a secondary cluster / cold storage:** a scheduled DistCp job replicates the curated (and optionally raw) zone to a geographically separate cluster or object store, satisfying the backup-reliability constraint independently of HDFS's own in-cluster replication.

4. How the Design Meets the Constraints

- Heterogeneous ingestion: each source uses its most efficient native connector, unified under one governed NiFi layer.
- Backup reliability: scheduled DistCp replication to a secondary cluster gives recoverability beyond HDFS's own replication.
- Minimal duplication: checksum/key-based deduplication in the metadata catalog, combined with incremental Sqoop/CDC imports, prevents redundant storage across all sources.

Question 5 — Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

1. Constraints Identified and Prioritized

- **Secure data movement:** data must be encrypted in transit and at rest, and every component must authenticate before it can read/write data.
- **High availability:** every layer of the stack — storage, resource management, coordination — must survive the loss of a single node without an outage.
- **NiFi/ETL integration:** the platform must plug cleanly into Apache NiFi (or an equivalent ETL tool) as the ingestion and orchestration front door.

2. End-to-End Architecture

The solution layers security (Kerberos + Ranger + TLS), high availability (NameNode/ResourceManager HA via ZooKeeper, plus a DR cluster), and NiFi-driven ingestion into a single coherent Hadoop ecosystem spanning ingestion, storage, processing, governance, and consumption.

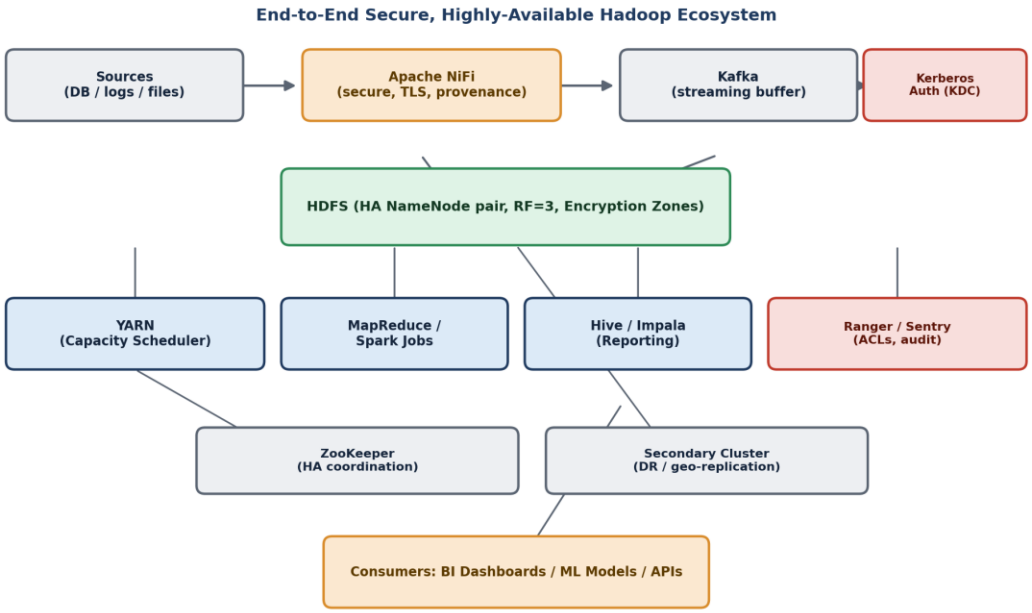

Figure 5: End-to-end secure, highly available Hadoop ecosystem integrated with Apache NiFi.

3. Layer-by-Layer Design and Justification

- **Ingestion — Apache NiFi with Kerberos-authenticated processors:** NiFi pulls from databases, logs, and files, and every processor authenticates via Kerberos before writing to HDFS/Kafka, so unauthenticated components cannot inject or read data.
- **Streaming buffer — Kafka (optional, for near-real-time feeds):** decouples fast-arriving sources from downstream batch processing and gives NiFi a durable, replayable buffer, improving both reliability and security posture (SASL_SSL-authenticated topics).
- **Kerberos KDC as the security backbone:** provides mutual authentication for every Hadoop daemon and client (NameNode, DataNode, NiFi, Hive, YARN), which is the standard mechanism for securing a Hadoop cluster end to end.
- **HDFS with HA NameNode pair and Transparent Data Encryption (TDE) zones:** the Active/Standby NameNode pair (via QJM + ZKFC) removes the metadata single point of failure; encryption zones keep sensitive media/records encrypted at rest with per-zone keys managed by a Key Management Server.
- **YARN with Capacity Scheduler and RM HA:** schedules MapReduce/Spark jobs with the same HA pattern as HDFS, ensuring processing continues even if the active ResourceManager fails.
- **Hive/Impala for reporting:** provides SQL-level access for analysts on top of curated HDFS data, with column/row-level access enforced by Ranger policies rather than raw HDFS permissions alone.
- **Apache Ranger (or Sentry) for centralized authorization and audit:** gives fine-grained, auditable access control layered on top of Kerberos authentication — satisfying secure data movement not just in transit but also in terms of who can act on the data once it lands.
- **ZooKeeper ensemble:** underpins HA for both HDFS (ZKFC) and YARN (RM election), and is itself deployed as an odd-numbered quorum (e.g., 3 or 5 nodes) so it tolerates its own node failures.
- **Secondary cluster for disaster recovery:** curated data and NiFi flow definitions are periodically replicated (DistCp / NiFi Site-to-Site) to a geographically separate cluster, extending high availability from 'survive a node failure' to 'survive a cluster/site failure'.

4. How the Design Satisfies All Constraints

- Secure data movement: Kerberos authenticates every component, TLS/SASL_SSL encrypts data in transit, and TDE encryption zones plus Ranger policies protect data at rest and control access.
- High availability: HA is applied at every layer — NameNode, ResourceManager, and ZooKeeper — and extended beyond the single cluster via DR replication to a secondary site.
- NiFi/ETL integration: NiFi sits at the front of the pipeline as the authenticated, auditable ingestion and orchestration layer, feeding both real-time (Kafka) and batch (HDFS) paths without bypassing security or HA controls.